



EFREI Paris

Formation *Big Data Framework*

Hubble Lakehouse

Rapport

Pipeline Big Data pour l'analyse des observations
du télescope spatial Hubble

Auteurs

Valentin Massonnière
Kevin Heugas

30 avril 2026

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Objectifs du projet	4
1.3	Livrables attendus	4
2	Périmètre fonctionnel	5
2.1	Profils utilisateurs	5
2.2	Hors-périmètre	5
3	Sources de données	6
3.1	Source 1 — MAST	6
3.2	Source 2 — NOAA SWPC	6
3.3	Choix des sources	6
3.4	Synchronisation temporelle des sources	6
4	Architecture	7
4.1	Vue d'ensemble	7
4.2	Briques techniques	7
5	Spécifications fonctionnelles	8
5.1	Ingestion	8
5.2	Transformations par couche	8
5.3	Datamarts	8
5.4	Restitution	8
6	Spécifications techniques	9
6.1	Volumétrie	9
6.2	Orchestration	9
6.3	Qualité et reprise	9
6.4	Bonus — déploiement conteneurisé	9
7	Modèle de données	10
7.1	Datamart Physicien	10
7.2	Datamart Chimiste	10
7.3	Datamart Biologiste	10
7.4	Datamart Ingénieur	10
8	Répartition des tâches	11
8.1	Valentin	11
8.2	Kevin	11
8.3	Tâches communes	11
9	Détail des livrables	12
9.1	Dépôt GitHub	12
9.2	Rapport	12
10	Explication technique	13
10.1	Stack locale et reproductibilité	13
10.2	Jobs Spark	13
10.3	API Flask	14
10.4	Déploiement Kubernetes	14
10.5	Pipeline CI/CD GitLab	15

11 Visualisation des résultats via Power BI	16
12 Conclusion	17

Note méthodologique. Ce rapport a d’abord été rédigé en brouillon par nous-mêmes. Une IA nous a aidé à nous corriger les fautes et reformuler certaines phrases, sans modifier les justifications ni ajouter de contenu. L’objectif est de gagner du temps de rédaction et de limiter les fautes.

1 Introduction

1.1 Contexte

Lancé en 1990 par la NASA et l'ESA, le télescope spatial Hubble (HST) constitue depuis plus de 30 ans l'un des instruments scientifiques le plus puissant de l'histoire de l'astronomie. Ses observations couvrent un large spectre, de l'ultraviolet au proche infrarouge, et alimentent des champs de recherche aussi variés que la cosmologie, l'étude des exoplanètes ou la physique stellaire.

L'ensemble des observations produites par Hubble est mis à disposition du public via deux archives principales : le *Mikulski Archive for Space Telescopes* (MAST) opéré par le STScI, et l'*ESA Hubble Science Archive* opéré par l'ESAC.

1.2 Objectifs du projet

Le projet vise à concevoir, implémenter et documenter une **architecture Lakehouse complète** suivant la logique *Médaille* (couches Bronze, Silver, Gold), capable de :

- **Ingérer** de manière fiable deux sources hétérogènes : un historique massif d'observations Hubble (batch) et un flux continu de météo spatiale (streaming) ;
- **Stocker** la donnée brute dans un système de fichiers distribué (HDFS) sans transformation préalable, garantissant rejouabilité et auditabilité ;
- **Nettoyer, typer et structurer** la donnée dans une couche intermédiaire (Hive / Parquet) servant de socle commun à l'ensemble des cas d'usage ;
- **Modéliser et matérialiser** dans une base relationnelle (PostgreSQL) quatre datamarts métier répondant à des problématiques scientifiques et techniques concrètes ;
- **Tracer** chaque étape du pipeline par des logs consultables depuis le dashboard Airflow.

1.3 Livrables attendus

Les livrables attendus sont :

- un **rapport écrit** décrivant l'architecture, les choix techniques, les jeux de données mobilisés et les résultats obtenus ;
- un **dépôt GitHub** regroupant l'ensemble du code source du pipeline.

2 Périmètre fonctionnel

2.1 Profils utilisateurs

Le projet répond à quatre problématiques posées par quatre métiers scientifiques.

Physicien. Quelles régions du ciel ont été sur-observées par Hubble, et quels sont les angles morts de la recherche ?

Chimiste. Quelles cibles ont été observées avec des filtres à bande étroite correspondant à des raies d'émission spécifiques ($H\alpha$, OIII, SII...)?

Biologiste. Quelles observations ont été acquises pendant une éruption solaire ou un événement de météo spatiale, et risquent d'être corrompues par les radiations ?

Ingénieur spatial. Comment évoluent les temps de calibration des instruments Hubble au fil du temps ?

2.2 Hors-périmètre

Le projet ne couvre pas :

- l'analyse du contenu des images FITS ; seules les métadonnées sont exploitées ;
- la prédiction (apprentissage automatique) ; les datamarts servent au reporting et à l'exploration ;

3 Sources de données

3.1 Source 1 — MAST

MAST est l'archive publique des observations Hubble, accessible via le [portail web](#) du STScI. Les métadonnées sont exportées au format CSV. Le fichier utilisé contient environ 500 000 lignes, décrites par le schéma CAOM (*Common Archive Observation Model*).

Pour répondre à l'obligation pédagogique d'avoir une base de données en source, le fichier CSV est chargé tel quel dans une table PostgreSQL dédiée au démarrage du projet. C'est cette table qui est ensuite consommée par le pipeline d'ingestion vers Bronze, et non le fichier CSV brut.

3.2 Source 2 — NOAA SWPC

NOAA SWPC publie en quasi-temps réel les indicateurs de météo spatiale : vent solaire, flux X-ray, indice planétaire Kp, éruptions solaires. Les données sont disponibles au format JSON sur des endpoints HTTP (services.swpc.noaa.gov/json/), la récupération se fait en mode streaming.

3.3 Choix des sources

MAST fournit un historique statique traité en batch. NOAA SWPC fournit un flux temps réel traité en streaming.

3.4 Synchronisation temporelle des sources

Les deux sources ne couvrent pas le même horizon temporel. MAST contient des observations qui s'étalent sur plus de trente ans, alors que les endpoints JSON de NOAA SWPC n'exposent que les indicateurs récents (quelques jours à quelques semaines selon le produit). Sans correction, le datamart Biologiste ne pourrait croiser les éruptions solaires qu'avec les rares observations Hubble réalisées pendant la fenêtre courante du flux SWPC.

Pour lever cette limite, les archives historiques de NOAA SWPC sont récupérées en amont puis chargées dans Bronze en complément du flux temps réel.

4 Architecture

4.1 Vue d'ensemble

L'architecture suit le patron Médaillon, organisé en trois couches successives : Bronze, Silver et Gold. Les données circulent des sources jusqu'aux outils de restitution en passant par chacune de ces couches. La figure 1 présente le pipeline complet.

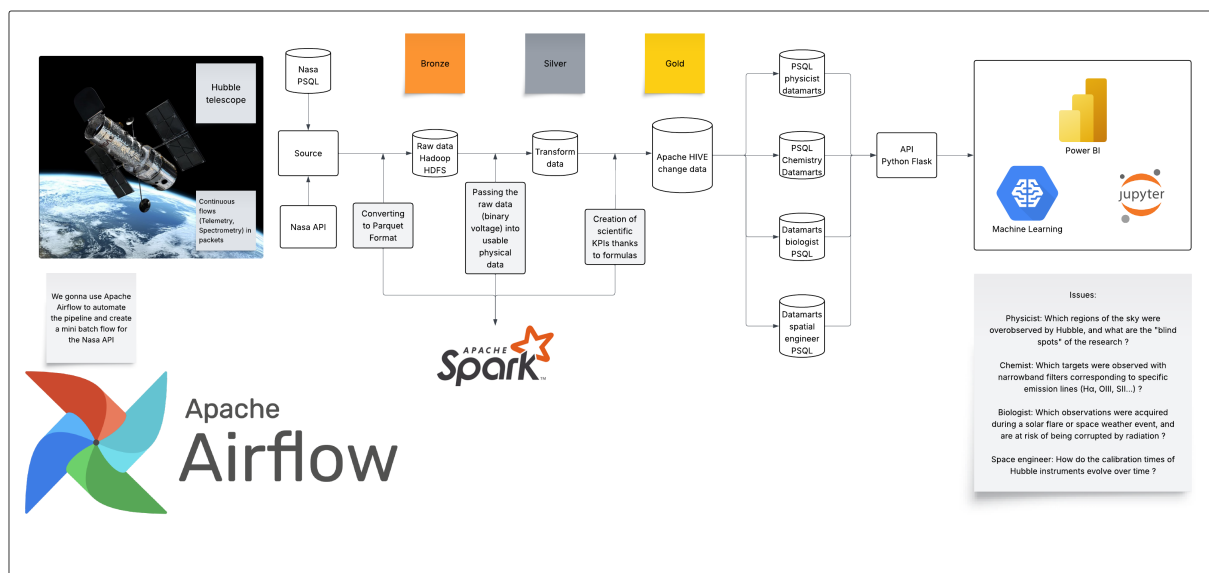


FIGURE 1 – Architecture Lakehouse du projet.

4.2 Briques techniques

L'architecture s'appuie sur les composants suivants :

- **PostgreSQL** : table source MAST en entrée, datamarts métier en sortie ;
- **HDFS** : stockage distribué des couches Bronze, Silver et Gold au format Parquet ;
- **Apache Spark** : ingestion NOAA SWPC (batch et streaming) et transformations Bronze → Silver → Gold ;
- **Apache Hive Metastore** : catalogue exposant les tables Gold en SQL standard ;
- **Apache Airflow** : orchestration des DAGs batch et streaming ;
- **API Flask** : exposition HTTPS des datamarts, protégée par JWT.

5 Spécifications fonctionnelles

5.1 Ingestion

MAST. La table PostgreSQL source, alimentée au démarrage du projet à partir de l'export CSV MAST, est lue en mode batch. Un job Airflow déclenche un script Python qui extrait son contenu et le dépose dans HDFS Bronze au format Parquet.

NOAA SWPC. Les endpoints JSON sont interrogés en mini-batch toutes les minutes. Un job Spark Structured Streaming lit les réponses HTTP et les écrit dans HDFS Bronze au format Parquet.

5.2 Transformations par couche

Bronze → Silver. Spark lit les fichiers Parquet bruts (MAST issu du dump PostgreSQL, archives et flux temps réel SWPC) et applique le typage des colonnes, le parsing des champs structurés (filtres, polygones, dates MJD vers UTC...) et la suppression des colonnes redondantes. Les apports archives et temps réel SWPC sont fusionnés et dédupliqués sur la clé temporelle pour produire une vue continue des indicateurs de météo spatiale. Les tables Silver sont écrites en Parquet sur HDFS, sans déclaration dans un catalogue.

Silver → Gold. Spark SQL agrège les tables Silver pour produire les indicateurs métier des quatre datamarts. Les résultats sont écrits en Parquet sur HDFS et déclarés comme tables externes dans le Hive Metastore.

Gold → Datamarts. Un dernier job Spark SQL, déclenché par un DAG Airflow distinct, lit chaque table `gold.*` via Hive et la matérialise dans PostgreSQL via JDBC, dans des schémas distincts de celui qui héberge la table MAST source. Cette étape isole la couche analytique (Hive) de la couche de service (Postgres), permettant à l'API de répondre sans dépendance à Spark.

5.3 Datamarts

Quatre datamarts PostgreSQL sont produits, un par profil utilisateur.

Physicien. Savoir quelles régions du ciel Hubble a beaucoup observées, et lesquelles sont restées dans l'ombre.

Chimiste. Retrouver les images prises à travers des filtres capables de révéler certains gaz dans l'espace, comme l'hydrogène, l'oxygène ou le soufre.

Biologiste. Repérer les observations prises pendant une éruption solaire, qui risquent d'être abîmées par les radiations.

Ingénieur spatial. Suivre l'usure des instruments de Hubble en regardant comment leur temps de calibration évolue au fil des années.

5.4 Restitution

Les quatre tables PostgreSQL sont exposées par une API REST écrite en Python avec Flask, accessible à l'adresse <https://efrei-bigdata-api.valentin-massonniere.ch>. Chaque datamart correspond à un endpoint dédié, qui retourne les enregistrements au format JSON. L'accès est protégé par jeton JWT : chaque requête doit présenter un token valide dans son en-tête **Authorization**. Les outils de visualisation et les scripts d'analyse consomment ces endpoints, sans accès direct à la base.

6 Spécifications techniques

6.1 Volumétrie

L'export MAST représente environ 500 000 observations, soit quelques centaines de méga-octets en Parquet. Le flux NOAA SWPC ajoute quelques kilo-octets par minute.

6.2 Orchestration

Apache Airflow pilote l'ensemble du pipeline via deux DAGs complémentaires.

full_pipeline. DAG manuel de backfill, déclenché ponctuellement (notamment au démarrage du projet). Il rétrocharge les archives NOAA SWPC sur la fenêtre voulue, ingère la table MAST source vers Silver, déduplique les apports SWPC, puis enchaîne pour chaque profil utilisateur les étapes Gold et la matérialisation du datamart en PostgreSQL. Le datamart Biologiste dépend des deux Silver (MAST et SWPC); les trois autres ne dépendent que de Silver MAST.

streaming. DAG planifié toutes les dix minutes. Il interroge les endpoints temps réel NOAA SWPC, met à jour Silver SWPC, recalcule la table Gold Biologiste et republie le datamart correspondant en PostgreSQL. C'est ce DAG qui maintient le datamart Biologiste à jour entre deux exécutions complètes du pipeline.

6.3 Qualité et reprise

Chaque couche est idempotente : un retraitement ne crée pas de doublons. Les enregistrements rejetés (champs manquants, valeurs incohérentes) sont conservés dans une table de rejet par couche.

6.4 Bonus — déploiement conteneurisé

En complément du périmètre commun, Valentin déploie le pipeline sur un VPS personnel. Chaque composant est conteneurisé avec Docker et orchestré par Kubernetes.

Les logs des jobs sont centralisés et consultables depuis le dashboard Airflow, accessible à l'adresse <https://efrei-bigdata-airflow.valentin-massonniere.ch> (utilisateur `invit`, mot de passe `inviter123`). Pour des raisons de sécurité du VPS, ce compte dispose d'un accès en lecture seule et ne peut pas déclencher de DAG manuellement.

7 Modèle de données

Chaque profil utilisateur est servi par une table PostgreSQL dédiée, alimentée par un job Spark SQL depuis la couche Silver. Les schémas des sources sont documentés ci-dessous :

7.1 Datamart Physicien

Quelles régions du ciel ont été sur-observées par Hubble, et quels sont les angles morts de la recherche ?

Colonne	Type	Description
ra_bin	float	Position est-ouest dans le ciel (sorte de longitude céleste).
dec_bin	float	Position nord-sud dans le ciel (sorte de latitude céleste).
nb_obs	int	Nombre de fois où cette zone du ciel a été photographiée.
total_exptime	float	Temps total passé à observer cette zone, en secondes.

7.2 Datamart Chimiste

Quelles cibles ont été observées avec des filtres à bande étroite correspondant à des raies d'émission spécifiques (H α , OIII, SII...)?

Colonne	Type	Description
target_name	string	Nom de l'objet observé (étoile, nébuleuse, galaxie...).
filter	string	Filtre coloré placé devant l'instrument pendant la prise.
emission_line	string	Gaz que ce filtre permet de mettre en évidence (hydrogène, oxygène, soufre...).
nb_obs	int	Nombre d'observations correspondantes.

7.3 Datamart Biologiste

Quelles observations ont été acquises pendant une éruption solaire ou un événement de météo spatiale, et risquent d'être corrompues par les radiations ?

Colonne	Type	Description
obs_id	string	Référence unique de l'observation.
t_start	timestamp	Date et heure du début de la prise.
t_end	timestamp	Date et heure de fin de la prise.
flare_class	string	Intensité de l'éruption solaire qui a eu lieu pendant la prise.
risk_score	int	Niveau de risque que l'image soit abîmée par les radiations.

7.4 Datamart Ingénieur

Comment évoluent les temps de calibration des instruments Hubble au fil du temps ?

Colonne	Type	Description
instrument	string	Nom de l'instrument de Hubble (caméra, spectrographe...).
year_month	date	Mois auquel la mesure correspond.
avg_calib_time	float	Temps moyen passé à régler l'instrument ce mois-là, en secondes.
trend_12m	float	Évolution sur les 12 derniers mois ; une valeur positive signifie que l'instrument met de plus en plus de temps à se régler.

8 Répartition des tâches

Le projet est mené en binôme. La répartition s'appuie sur les appétences techniques de chacun.

8.1 Valentin

- Mise en place et maintenance du dépôt GitHub.
- Déploiement de l'infrastructure (HDFS, Spark, Hive, PostgreSQL, Airflow) sur un VPS personnel via Docker et Kubernetes (bonus).
- Ingestion du flux NOAA SWPC via Spark Structured Streaming.
- Transformations Silver et Gold liées aux événements de météo spatiale.
- Datamarts Biologiste et Ingénieur.
- Coordination générale du projet.

8.2 Kevin

- Ingestion batch de l'export MAST via Python et Airflow.
- Transformations Silver des observations Hubble (typage, parsing des filtres, conversion MJD vers UTC...).
- Modélisation des tables Hive de la couche Silver.
- Datamarts Physicien et Chimiste.

8.3 Tâches communes

- Choix techniques structurants (formats, schémas Gold, conventions de nommage).
- Rédaction du rapport.

9 Détail des livrables

9.1 Dépôt GitHub

Le dépôt est hébergé à l'adresse github.com/TheValll/efrei-projet-big-data et miroiré sur GitLab pour la pipeline CI/CD. Il regroupe l'ensemble des artefacts du projet, organisés comme ceci :

- **ingestion/** : export CSV MAST et script Python de chargement vers la table PostgreSQL source ;
- **spark/** : jobs Spark d'ingestion NOAA SWPC (archives et temps réel) et de transformation Silver → Gold → datamart pour chaque profil ;
- **dags/** : DAGs Airflow **full_pipeline** (backfill manuel) et **streaming** (rafraîchissement Biologiste toutes les dix minutes) ;
- **api/** : API Flask en MVC (**controllers/**, **models/**, **app.py**) exposant les datamarts ;
- **bruno/** : collection Bruno prête à l'emploi (environnements local et prod, login et endpoints datamarts) ;
- **infra/** : Dockerfiles (Airflow, API, Spark, Hive Metastore), configurations Hadoop / Hive / Postgres, et manifeste Kubernetes **k8s.yaml** ;
- **dumps/** : dump SQL de la base **datamart** pour rejouer un état local ;
- **docs/** : rapport **L^AT_EX** et documentation technique complémentaire ;
- **docker-compose.yml**, **Makefile**, **.gitlab-ci.yml**, **README.md** : stack locale, raccourcis (**make up/down/dump/restore**), pipeline CI/CD et guide de démarrage.

9.2 Rapport

Ce document intègre le cahier des charges et sera complété au fil du projet par :

- l'architecture finale réellement implémentée ;
- les choix techniques et leurs justifications ;
- les résultats obtenus pour chaque datamart, avec exemples ;
- les difficultés rencontrées et les pistes d'amélioration.

10 Explication technique

Cette section détaille l'implémentation concrète du projet, depuis la stack locale jusqu'au déploiement en production.

10.1 Stack locale et reproductibilité

L'intégralité du projet tourne en local via un seul fichier `docker-compose.yml`, sans dépendance à un cluster ou à un service managé. Huit services y sont déclarés : PostgreSQL, le NameNode et un DataNode HDFS, un service de bootstrap PostgreSQL, le Hive Metastore, un master et un worker Spark, Airflow et l'API Flask.

Images. Quatre images sont construites à partir du dossier `infra/` : `Dockerfile.airflow` (Airflow 3.0 avec client Spark et providers), `Dockerfile.api` (API Flask servie par Gunicorn), `Dockerfile.spark` (Spark 3.5 avec connecteurs Postgres et Hadoop) et `Dockerfile.hive-metastore` (Hive Metastore standalone). Les autres composants utilisent des images publiques : `postgres:16` et les images Hadoop `bde2020/hadoop-namenode` et `bde2020/hadoop-datanode`.

Bootstrap PostgreSQL. Une instance PostgreSQL unique héberge à la fois les métadonnées Airflow, le metastore Hive, la table source MAST et les datamarts. Au premier démarrage, un service `postgres-bootstrap` idempotent exécute un script Bash qui crée les rôles `airflow` et `hive` ainsi que les quatre bases `airflow_metadata`, `nasa_db_raw` (table source MAST), `datamart` (datamarts Gold) et `hive_metastore`. Les autres services attendent sa terminaison via `depends_on` avec la condition `service_completed_successfully`.

Persistance et réseau. Trois volumes nommés (`postgres-data`, `hadoop-namenode`, `hadoop-datanode`) garantissent la persistance des données entre redémarrages. Les services communiquent sur le réseau Docker par défaut, en utilisant les noms de services comme noms d'hôtes (`postgres`, `namenode`, `spark-master`, `hive-metastore`...). Les UIs sont exposées sur l'hôte : Airflow sur 8080, Spark sur 8081, HDFS NameNode sur 9870, API sur 5000 et PostgreSQL sur 5432.

Configuration par variables d'environnement. Tous les paramètres sensibles (mots de passe Postgres, Airflow, API, secret JWT) sont injectés depuis un fichier `.env` non versionné. Un `.env.example` fournit des valeurs de développement par défaut et documente les clés attendues.

Makefile. Quatre cibles couvrent le cycle de vie local : `make up` (build et démarrage de la stack), `make down` (arrêt sans suppression des volumes), `make dump` (export de la base `datamart` vers `dumps/datamart.sql`) et `make restore` (rejeu d'un dump local).

10.2 Jobs Spark

Le dossier `spark/` regroupe douze scripts PySpark répartis en quatre étapes : ingestion NOAA SWPC vers Bronze, Bronze → Silver, Silver → Gold et Gold → datamart PostgreSQL.

Ingestion NOAA SWPC vers Bronze. Deux scripts (live et archive) écrivent dans le même répertoire Bronze sous un schéma unifié (`endpoint`, `event_time`, `record`, `ingested_at`) et appliquent la même stratégie de fusion : union avec l'existant puis déduplication par fenêtre sur (`endpoint`, `event_time`). Le live interroge quatre endpoints JSON (Kp, X-ray, vent solaire, éruptions) ; l'archive parse les rapports texte annuels GOES X-Ray.

Bronze → Silver. `silver_mast.py` lit la table source via JDBC, type les colonnes et convertit les dates MJD en `timestamp` UTC. `silver_noaa.py` parse le JSON Bronze, extrait la classe X-ray par *regex* et déduplique les apports archives et live sur (`event_start`, `xray_class`).

Silver → Gold. Les quatre jobs déclarent les tables externes Silver dans Hive, exécutent une requête Spark SQL d'agrégation et publient le résultat comme table externe `gold.*`. Les spécificités métier :

- **Physicien** : binning du ciel par degré (FLOOR sur `s_ra`, `s_dec`) ;
- **Chimiste** : jointure avec une table de référence des filtres à bande étroite ;
- **Biologiste** : jointure d'intervalles temporels MAST × éruptions, calcul d'un `risk_score` dérivé de la classe X-ray ;
- **Ingénieur** : agrégation mensuelle par instrument sur les observations de calibration, tendance via LAG sur 12 mois.

Gold → datamart. Les quatre scripts `datamart_*.py` lisent chacun leur table `gold.*` via Hive et l'écrivent en `overwrite` dans la base `datamart` via JDBC.

10.3 API Flask

L'API est servie par Gunicorn et organisée en MVC : `app.py` (factory et configuration), `controllers/` (blueprints HTTP) et `models/` (requêtes SQL). Trois blueprints sont enregistrés : `/health`, `/auth` et `/datamarts`.

Authentification. `/auth/login` compare les identifiants envoyés à deux comptes (`admin`, `viewer`) configurés par variables d'environnement, puis émet un `access_token` (1 h) et un `refresh_token` (7j) via *flask-jwt-extended*. `/auth/refresh` renouvelle l'`access_token` à partir du refresh.

Endpoints datamarts. Quatre routes GET `/datamarts/{profil}` renvoient les lignes du datamart correspondant en JSON. Toutes sont protégées par `@jwt_required()` et acceptent une pagination `limit / offset` ; les profils Biologiste et Ingénieur acceptent en plus un filtre temporel `from / to`.

Accès PostgreSQL. Les modèles utilisent un pool `psycopg2` (`DBPool`) initialisé au démarrage avec `DATAMART_DSN`, et un curseur `RealDictCursor` pour sérialiser directement en JSON. Une collection Bruno (`bruno/`) fournit les requêtes prêtes à l'emploi pour les environnements local et prod.

10.4 Déploiement Kubernetes

L'ensemble de la stack est redéployé sur un cluster Kubernetes hébergé sur un VPS personnel, dans le namespace `efrei-big-data`. Le fichier `infra/k8s.yaml` regroupe l'intégralité des ressources : huit `Deployment` (Postgres, Airflow, API, NameNode, DataNode, Spark master, Spark worker, Hive Metastore), leurs `Service` associés, trois `PersistentVolumeClaim` pour les données persistantes et un `ConfigMap` d'initialisation Postgres.

Images. Les images custom (Airflow, API, Spark, Hive Metastore) sont tirées depuis le registry GitLab via un `imagePullSecret` `regcred`. Postgres et Hadoop utilisent les mêmes images publiques qu'en local.

Secrets. Un `Secret` `efrei-secrets` centralise les mots de passe Postgres et Airflow ; il est recréé à chaque déploiement par la pipeline CI/CD à partir de variables protégées GitLab.

Exposition HTTPS. Deux Ingress routent `efrei-bigdata-airflow.valentin-massonniere.ch` et `efrei-bigdata-api.valentin-massonniere.ch` vers les services internes. Les certificats TLS sont émis automatiquement par *cert-manager* avec l'issuer `letsencrypt-prod`.

10.5 Pipeline CI/CD GitLab

Le dépôt est miroiré sur GitLab pour profiter de son CI/CD intégré. Le fichier `.gitlab-ci.yml` déclare deux étapes (`build` puis `deploy`) qui ne s'exécutent que sur `main`.

Build. Quatre jobs construisent en parallèle les images Airflow, API, Spark et Hive Metastore avec Docker-in-Docker, puis les poussent sur le registry GitLab sous deux tags : le SHA court du commit et `latest`.

Deploy. Un job final utilise l'image `dtzar/helm-kubect1` pour reconstruire un `kubeconfig` à partir d'une variable CI/CD encodée en base64, recréer les secrets (`regcred` pour le registry, `efrei-secrets` pour les mots de passe), appliquer `infra/k8s.yaml` et déclencher un `rollout restart` sur les huit déploiements pour qu'ils retirent les nouvelles images.

Variables sensibles. Mots de passe (Postgres, Airflow), `kubeconfig` et identifiants registry sont stockés en variables protégées GitLab et n'apparaissent jamais dans le dépôt ni dans les logs CI/CD.

11 Visualisation des résultats via Power BI

12 Conclusion

Hubble Lakehouse

Pipeline Big Data pour l'analyse des observations du télescope spatial Hubble

Signatures

Valentin Massonnière

Efrei Paris

M1 Data Engineering & AI — DE3

Kevin Heugas

Efrei Paris

M1 Data Engineering & AI — DE3

Fait à Paris, le 30 avril 2026.